

FOFO: A Benchmark to Evaluate LLMs' Format-Following Capability

Congying Xia^{1,*}, Chen Xing^{1,*}, Jiangshu Du², Xinyi Yang¹,
Yihao Feng¹, Ran Xu¹, Wenpeng Yin³, Caiming Xiong¹

¹Salesforce Research

²University of Illinois at Chicago

³Pennsylvania State University

{c.xia,x.yang, yihao.feng, ran.xu, cxiong}@salesforce.com, xingchen1113@gmail.com,
jdu25@uic.edu, wenpeng@psu.edu

Abstract

This paper presents FOFO, a pioneering benchmark for evaluating large language models' (LLMs) ability to follow complex, domain-specific formats, a crucial yet underexamined capability for their application as AI agents. Despite LLMs' advancements, existing benchmarks fail to assess their format-following proficiency adequately. FOFO fills this gap with a diverse range of real-world formats and instructions, developed through an AI-Human collaborative method. Our evaluation across both open-source (e.g., Llama 2, WizardLM) and closed-source (e.g., GPT-4, PALM2, Gemini) LLMs highlights three key findings: open-source models significantly lag behind closed-source ones in format adherence; LLMs' format-following performance is independent of their content generation quality; and LLMs' format proficiency varies across different domains. These insights suggest the need for specialized tuning for format-following skills and highlight FOFO's role in guiding the selection of domain-specific AI agents. FOFO is released here. ¹

1 Introduction

Large language models (LLMs) show great promise in automating diverse tasks, from medical (Thirunavukarasu et al., 2023; Shah et al., 2023; Clusmann et al., 2023; Tang et al., 2023) and legal data analysis (Cui et al., 2023; Jiang and Yang, 2023; Fei et al., 2023; Guha et al., 2023) to daily assistance with activities like reservations (Ma et al., 2023; Gao et al., 2023; Xi et al., 2023; Muthusamy et al., 2023). For LLMs to effectively assist humans, their crucial ability lies in following instructions (Ouyang et al., 2022; Chung et al., 2022; Wang et al., 2022; Wei et al., 2021). In tasks requiring them to act as AI agents, such as organizing medical records or generating KPI reports, precise adherence to specified formats given by humans is

essential (Xi et al., 2023). Without this capability, the practicality of employing LLMs in such roles diminishes significantly.

However, prior evaluation benchmarks of LLMs fall short in assessing their format-following capabilities. On one hand, mainstream instruction-following benchmarks, e.g., AlpacaEval (Dubois et al., 2023) and MT-Bench (Zheng et al., 2023a), are in the open question-answering/chatting style, assessing the correctness of LLM responded content (therefore, we refer to those as CONTENT-FOLLOWING benchmarks) without explicitly considering their format-following capability. State-of-the-art LLMs, whether closed or open-source, increasingly demonstrate comparable performance on these CONTENT-FOLLOWING benchmarks. On the other hand, benchmarks evaluating AI agents (Yang et al., 2023; Zhou et al., 2023b; Shridhar et al., 2020) in specific environments typically prioritize the overall success rate in completing test tasks. Since there are many factors that can affect final success rates, such as the model's grounding and reasoning capability in the specific environment, the final success rates cannot directly gauge LLMs' format-following proficiency. Consequently, although we observe LLMs performing notably worse than humans in certain domains (Zhou et al., 2023b), it remains unclear if this discrepancy is partly attributed to format-following limitations. Exploring enhancements in LLMs' format-following may potentially pave the way for further improvements in their role as AI agents.

In this work, we take the lead to build FOFO, a benchmark specifically designed for assessing LLMs' FORMAT-FOLLOWING capabilities. We equip FOFO with two shining features: *a wide coverage of real-world domain-specific formats (such as HL7-CDA format (Dolin et al., 2001) in Healthcare) in various domains* that are likely to embrace LLMs as agents, and *complicated and practically occurring format-oriented instructions* so that the

*Indicates Equal Contribution

¹<https://github.com/SalesforceAIResearch/FoFo>

LLMs can be tested with real-world complex context. To achieve these, we implement an AI-Human collaborative strategy for developing FOFO, featuring a structured hierarchical layout through a three-step process: i) identification and collection of domains and subdomains that AI Agents can assist humans; ii) creation of data formats specific to each subdomain; iii) generation of format-oriented instructions (FORMAT-INSTRU) that include complex format requirements and real-world context for each (domain, subdomain, data format).

In experiments, we test a diverse set of closed-source LLMs (e.g., GPT-4 (OpenAI, 2023b), PALM2 (Anil et al., 2023), Gemini (Google, 2023)) and open-source LLMs, such as Llama 2 (Touvron et al., 2023), WizardLM (Xu et al., 2023), Mistral (Jiang et al., 2023), etc., on these FORMAT-INSTRU. We use GPT-4 as the main judge and human evaluation is conducted to ensure a high GPT4-Human agreement. The following three observations are highlighted:

- For most of the LLMs tested, the rankings of their performance on format-following do not consistently align with their rankings on content-following benchmarks (e.g., AlpacaEval). In other words, LLMs achieving good content-following performance might perform poorly on format-following and vice versa.
- Regarding format-following, open-source LLMs lag notably behind closed-source models like GPT-3.5 and Google’s Gemini, in spite of the fact that they all exhibit similar performances on content-following benchmarks.
- The format-following capability of LLMs varies widely across domains. Even LLMs with similar overall benchmark accuracy may demonstrate significant differences in accuracy within specific domains.

These observations suggest two key points: i) The format-following capacity of LLMs appears independent of other capabilities and may necessitate specialized alignment fine-tuning beyond the conventional instruction-tuning used in training open-source LLMs. ii) Format-following capacity is not universally transferable across domains, highlighting the potential utility of our benchmark as a guiding and probing tool for selecting domain-specific AI agent foundation models.

To our knowledge, this is the first work that breaks down LLMs’ instruction-following behavior

into content-following and format-following, and benchmarks the evaluation of format-following capacity. This study contributes valuable insights into comprehending LLMs’ capabilities and offer guidance in selecting LLMs, particularly for domain-specific agent development. Our FOFO dataset will be public released.

2 Related Work

CONTENT-FOLLOWING benchmarks. There are various existing efforts that build evaluation data sets to try to assess LLMs’ general problem-solving capability through conversations. Among them, MMLU dataset (Hendrycks et al., 2020) is collected to measure knowledge and problem solving capabilities of LLMs in different knowledge domains such as elementary mathematics and US history. The performance of LLMs on MMLU is measured by the accuracy of their selected answers from multiple options. AlpacaEval (Dubois et al., 2023) and MT-Bench (Zheng et al., 2023a), on the other hand, collect open-ended questions across different domains without providing concrete reference answers. They rely on LLMs such as GPT-4 to conduct automatic evaluations on the target LLM’s answers. Except for these general benchmarks, there are also evaluation data sets specifically focusing on assessing LLM answers’ truthfulness (Si et al., 2023; Lin et al., 2022) and safety (Bhardwaj and Poria, 2023; Zhang et al., 2023; Zheng et al., 2023b).

FORMAT-FOLLOWING benchmarks. In the past few months, several instruction-following benchmarks are recently curated to contain a small sub-set of test cases relevant to format following (Zhou et al., 2023a; Chen et al., 2024), such as generating data following JSON format, or following format requirements such as numbers of bullet points/paragraphs (Zhou et al., 2023a). Compared to such format-following sub-sets that only covers a handful of generic formats such as JSON, our benchmark covers more diverse and domain-specific format requirements and each test example in our benchmark comes with complicated combined requirements and domain-specific context. Therefore, we empirically find that our format-following benchmark is harder for existing LLMs and can unveil performance discrepancy across different domains, compared to the format-following sub sets in these existing benchmarks.

Domains	Subdomains
Healthcare	Medical Diagnostics; Medical Treatment; Patient Care Management; Clinical Trial Analysis; Pharmaceuticals
Finance	Fraud Detection; Algorithm Trading; Personalized Financial Advice; Risk management; Regulatory Compliance
Technology and Software	Web Design; Programming; UI/UX Design; Data Analysis; Testing
Commerce and Manufacturing	E-commerce Personalization; Manufacturing Process Optimization; Inventory and Supply Chain Management; Quality Control; Smart Logistics and Route Optimization
Customer Relationship Management (CRM)	Customer Service; Sales Forecasting; Recruitment Assistants; Project Management; Lead Scoring
Marketing	Consumer Behavior Analysis; Advertising Campaign Optimization; Content Curation and Creation; Social Media Management; Search Engine Optimization
Scientific Research and Development	Mathematical Research; Physics; Chemistry and Biological Sciences; Environmental Sciences and Climate Change; Space Exploration
Education	Adaptive Learning Platforms; Intelligent Tutoring Systems; Automated Grading Systems; Education Data Analysis; Language Learning
Legal	Contract Review and Analysis; Legal Document Automation; Legal Research; Predictive Legal Analytics; Intellectual Property (IP) Management
Arts and Entertainment	Music Composition; Film Scriptwriting; Visual Art Creation; Video Game Development; Sports Analytics and Performance

Table 1: Full list of domains and subdomains.

3 Dataset Curation and Evaluation

3.1 FoFo Construction

The construction of FoFo unfold in three steps: i) collecting domains & subdomains, ii) gathering domain-specific data formats (each format is expressed by a name), and iii) generating FORMAT-INSTRU for each (domain, subdomain, format) triplet. Next, we elaborate on each step in detail.

Collecting domains & subdomains. To collect the domain and subdomain list for our benchmark, we adopt an iterative methodology that synergizes human expertise with the advanced capabilities of GPT-4. This process starts with an initial list of subdomains identified by domain experts. These subdomains, including but not limited to “Web Design”, “Programming”, and “Medical Diagnostics”, represent areas where LLM agents have shown significant potential. This preliminary list acted as a foundation for the next phase, in which we steer GPT-4 to extend this collection by the following prompt:

Can you list the domains that AI agents might help? These are some examples: Web Design, Programming, and Medical Diagnostics. Expand beyond these fields to cover all potential domains.

Following the expansion of subdomains, we utilize GPT-4 to summarize these subdomains with an instruction like: “Summarize these domains into super domains”. Subsequently, human experts conduct a thorough review of these proposed domains

and subdomains. Any domain or subdomain misaligned with our benchmark’s objectives triggers a reiteration of GPT-4 to regenerate and fill in a proper domain/subdomain, followed by a subsequent expert review. This iterative cycle of generation, review, and refinement, bridging human intellectual finesse with AI efficiency, results in a well-defined list of domains and subdomains. In the end, we obtain 10 domains with each having 5 subdomains, as listed in Table 1.

Gathering domain-specific data formats. For each identified domain and subdomain, we ask GPT-4 to generate five human-understandable text data formats that an AI agent is likely to encounter. We restrict the output data formats to text-only, ensuring they are producible by LLMs. Additionally, we instruct GPT-4 to skip generic formats in this generation process to prevent the production of similar data formats across different subdomains, such as TXT, CSV, and XML. The concrete prompt is:

Please give 5 human-understandable text data formats that an AI agent in the domain of { domain } -> { subdomain } would likely encounter as its required output formats during its interaction with humans. Note that only text data format should be provided. The data formats should also be as domain-specific as possible. Generic formats such as TXT, CSV, JSON, XML, etc, shouldn’t be included. An example of a piece of data of a specific format should be provided after the name of each format.

After generating these data formats, we engage human experts to assess the quality of each format. We either regenerate or remove the data format if it does not align with the requirements described above. Some examples of domain-specific data formats are listed in Table 2. For the full list, please check the Appendix A.6. From Table 2 we can see that among the generated domain-specific formats, some are existing formats that have relatively fixed and commonly acknowledged configurations, such as MathML and Maple, etc. Others are format names that would require further detailed specifications, such as Manufacturing Reports Format and Prescription Format, etc. For both of the two categories of generated formats, we will include enough detailed format specifications in our next step of generating test prompts. After fixing these domain-specific formats, we also add back prevalent/universal data formats applicable across all domains/subdomains, including JSON, XML, CSV, Markdown, and YAML.

FORMAT-INSTRU generation. The final phase of developing the FOFO benchmark entails the utilization of GPT-4 to devise FORMAT-INSTRU spanning a wide array of domains, subdomains and target formats. The following prompt is employed:

You are a helpful agent. Please write an instruction for an AI agent in the domain of { domain } -> { subdomain }. The task of the instruction should be detailed and complicated content generation in the given domain. The task should require the output to strictly adhere to a { format } format with specific configurations. If the format name is not specific enough, please give concrete illustrations of the specific format requirements to follow. Please try your best to give detailed dummy context/data required in the prompt when necessary. If you cannot give all necessary dummy data, please mention in the instruction that the AI agent is allowed to make up the data required and improvise on ungiven details. Your response should only contain the instruction or question, without any preliminary or concluding statements.

These FORMAT-INSTRU are designed with a strict requirement for the outputs to conform precisely to specified data formats. In instances where the name of the data format lacks specificity, we direct GPT-4 to provide detailed examples illustrating the exact format specifications that must be adhered to. We request GPT-4 to construct tasks of con-

Generate a comprehensive medical prescription for a 55-year-old patient diagnosed with Type 2 Diabetes Mellitus and Hypertension. The prescription should adhere to the Prescription Format Standard (PFS- 2021) specified below:

1. Patient Information:

- Full Name: John A. Doe
- Age: 55
- Sex: Male
- Weight: 90 kg
- Height: 175 cm
- Allergies: Penicillin

2. Diagnosis:

- Primary Diagnosis: Type 2 Diabetes Mellitus
- Secondary Diagnosis: Hypertension

3. Prescription Date: [Today's date]

4. Medications:

- Itemize each medication with the following details:
 - a. Generic Name
 - b. Brand Name (if applicable)
 - c. Strength
 - d. Form (tablet, capsule, injection, etc.)
 - e. Dosage
 - f. Administration route (oral, IV, etc.)
 - g. Frequency and duration of intake
 - h. Special instructions (e.g., taken with food, on an empty stomach, etc.)

5. Lifestyle Recommendations:

- Include at least three recommendations relevant to the diagnosis.

6. Lab Tests:

- List any lab tests required before the next visit.

7. Refills:

- Specify the number of refills for each medication.

8. Physician Information:

- Full Name: Dr. Emily R. Smith
- Medical License Number: 123456
- Specialty: Endocrinology
- Contact Number: (555) 123-4567
- Clinic Address: 123 Healthy Way, Wellness City, HC 67890

9. Follow-up:

- Date and Time for next appointment (if applicable).

Ensure the following PFS-2021 configuration requirements are met:

- Align patient information and diagnosis to the left
- Medications should be listed in a table format with clear column headings
- Lifestyle recommendations should be in bullet points
- Lab Tests and Refills should be in separate, clearly labeled sections

Please improvise any additional details required that have not been explicitly provided, ensuring the generated content aligns with the patient's condition and common medical practice.

Figure 1: An FORMAT-INSTRU example when “Domain” = “Healthcare”, “Subdomain” = “Medical Treatment”, and “Format” = “Prescription Format”. It has many detailed format requirements and missing one single format requirement would cause the target LLM to fail on this example, making our benchmark harder to ace for LLMs.

siderable complexity, ensuring that the generated FORMAT-INSTRU are sufficiently detailed. Furthermore, we instruct GPT-4 to include comprehensive dummy context or data, deemed essential for the

Domains	Healthcare	Commerce and Manufacturing	Scientific R&D
Subdomains	Medical Treatment	Manufacturing Process Optimization	Mathematical Research
Data Formats	Medical Reports	Manufacturing Reports Format	LaTeX
	Prescription Formats	Bill of Materials	MathML
	SOAP Notes	Work Instruction Format	SageMath Notebooks
	Discharge Summary	Standard Operating Procedure	Maple
	Clinical Trial Protocols	Production Scheduling Format	MATLAB scripts

Table 2: Examples of domain specific data formats under different domains and subdomains. Among the generated domain-specific formats, some have relatively fixed and commonly acknowledged configurations, such as MathML and Maple, etc. Others are format names that would require further detailed specifications, such as Manufacturing Reports Format and Prescription Format, etc. For both of the two categories of generated formats, we generate enough detailed format specifications in our next step of generating test prompts.

completion of these tasks. This approach not only fosters a rigorous evaluation framework for LMs but also simulates a diverse array of real-world scenarios, thereby enhancing the benchmark’s relevance and applicability. Additionally, we engage human experts to verify each generated instruction, with the authority to edit, remove, or regenerate the instruction as necessary to maintain the quality of FORMAT-INSTRU.

An illustrative FORMAT-INSTRU is depicted in Figure 1, situated in the *Healthcare* domain, *Medical Treatment* subdomain, and *Prescription Format*. The instruction requires generating a comprehensive prescription for a patient with Type 2 Diabetes Mellitus and Hypertension, challenging LLMs to adhere to intricate specifications. We can see from this example that it has many detailed format requirements, such as the content of each section, how to itemize generated content under each section, detailed FS-2021 configurations, what content to replace and what not to replace, etc. Missing one requirement would cause the target LLM to fail on this example. All of our generated FORMAT-INSTRU are of similar complexity level. More examples are illustrated in Appendix A.4.

After these three steps, our FoFo dataset is finalized, with detailed statistics in Table 3. The average length of FORMAT-INSTRU is the number of characters to maintain consistency with AlpacaEval.

3.2 Evaluation Metric

Given each FORMAT-INSTRU and corresponding LLM’s response, we model the evaluation as a binary classification problem with the further requirement of generating a detailed explanation for its assessment. GPT-4 is used as the evaluator, akin to methodologies employed in AlpacaEval and MT-Bench, to minimize annotation efforts. The struc-

Attributes	Number
# Domains	10
# Subdomains	50
# Data Formats	248
# FORMAT-INSTRU	494
Average Length of FORMAT-INSTRU	2,908

Table 3: FoFo statistics.

ture of the evaluation prompt is detailed in Figure 7 in Appendix, where we outline how GPT-4 is directed to assess the fidelity of responses from different LLMs to predefined format requirements. Missing a single specific format requirement among all the requirements in a FORMAT-INSTRU would lead to failing on this prompt, making our benchmark harder to ace for LLMs. Our benchmark specifically focuses on assessing the ability of LLMs to comply with given format guidelines, underscoring the importance of format over content.

4 Experiments

In this section, we present our empirical results and analysis of FoFo. To verify FoFo’s effectiveness on serving as a format-following benchmark of LLMs, we firstly select top-performing LLMs from both the closed-source and open-source world and evaluate them on FoFo. Specifically, we are more interested in middle-sized LLMs that have shown similar performances to GPT-3.5 and GPT-4 on existing content-following benchmarks such as MT-Bench and AlpacaEval because such LLMs are currently most widely-used.

To evaluate each open-source LLM, we employ its official prompt format to conduct generation given each test prompt in FoFo. During generation, we use sampling and set the temperature as 0.7 for all models for fair comparison. We also

	Model	FORMAT-FOLLOWING	CONTENT-FOLLOWING	
		FoFo	MT-Bench	AlpacaEval
Open-source	Vicuna 13B V1.3 (Chiang et al., 2023)	22.74	6.39	82.11
	WizardLM 13B V1.1 (Xu et al., 2023)	27.00	6.76	86.32
	Vicuna 13B V1.5-16k (Chiang et al., 2023)	27.08	6.92	-
	Openchat V3.2-super (Wang et al., 2023)	31.22	7.19	89.50
	Llama 2 7B Chat (Touvron et al., 2023)	45.44	6.27	71.37
	Mistral 7B Instruct V0.1 (Jiang et al., 2023)	46.91	6.84	92.78
	Llama 2 13B Chat (Touvron et al., 2023)	53.28	6.77	81.09
	WizardLM 13B V1.2 (Xu et al., 2023)	63.54	7.2	89.17
	Zephyr 7B Beta (Tunstall et al., 2023)	64.12	7.34	90.60
Closed-source	GPT-3.5 (OpenAI, 2023a)	80.66	8.32	93.42
	Gemini Pro (Google, 2023)	80.25	-	79.66
	PaLM 2 for Text 32k (Anil et al., 2023)	83.72	-	-
	GPT-4 (OpenAI, 2023b)	91.17	9.18	95.28

Table 4: Main Results. The source of the models can be found in Appendix A.1.

set the max new tokens to generate as 5120 for all models. After generation, we evaluate each LLM’s format-following accuracy with GPT-4 as judge, as illustrated in Section 3.2.

4.1 Main Results

Table 4 presents the format accuracy of all selected LLMs on FoFo. To better see the performance discrepancy of the same LLM on different benchmarks, we also list each LLM’s performance on AlpacaEval and MT-Bench in Table 4.

The first observation we have from Table 4 is that performance rankings of LLMs on FoFo is not consistent with their rankings on content-following evaluation benchmarks. For example, Openchat V3.2-super and WizardLM 13B V1.2 have similar performance on both AlpacaEval (around 89%) and MT-Bench (around 7.2). While their format accuracy on FoFo has more than 30 points gap (31.22% vs 63.54%). Gemini-pro has lower performance compared to GPT-3.5 on AlpacaEval while their format accuracy on FoFo is similar. We can see similar patterns when viewing FoFo results side-by-side with those on AlpacaEval 2.0.

Second, we have found that closed-source models such as GPT-3.5 and Gemini Pro, significantly outperform open-source models. Closed-source models all have above 80% of format accuracy while open-source models reach only below 70% of format accuracy. This performance gap is much more significant compared to their performance gap on existing content-following benchmarks such as AlpacaEval. For example, Mistral 7B Instruct V0.1 achieves 92.78% of accuracy on AlpacaEval,

comparable with GPT-3.5’s 93.42%. While it’s format accuracy on FoFo is 46.91%, much lower compared to GPT-3.5’s 80.66%. Other open-source models also show similar performance pattern.

These two observations suggest that format-following capability is independent of other capabilities of LLMs reviewed by existing content-following evaluation benchmarks. It might require tailored alignment fine-tuning with specific data beyond regular instruction-tuning that are widely used for fine-tuning open-sourced LLMs.

4.2 Result Analysis

Domain Analysis. When analyzing empirical results on FoFo, we have also noticed that the format-following capability of LLMs may vary a lot across different domains. For two LLMs that give similar final accuracy on FoFo, their accuracy on different domains can be very different. Figure 2 shows two examples. Figure 2(a) shows the format-following accuracy comparison between Mistral 7B V0.1 and Llama 7B Chat across different domains. Their final performance on FoFo are very similar and both around 46%. While we can see from Figure 2(a) that Llama 7B Chat performs significantly worse on Scientific Research and Development domain and performs much better on Education domain. Similarly, Zephyr 7B Beta and WizardLM 13B V1.2 have similar final performance on FoFo (around 64%) while Figure 2(b) shows they have their own expertise on different domains. It indicates that format-following capacity is not generalizable across domains, possibly due to domain-specific formats, such as SOP

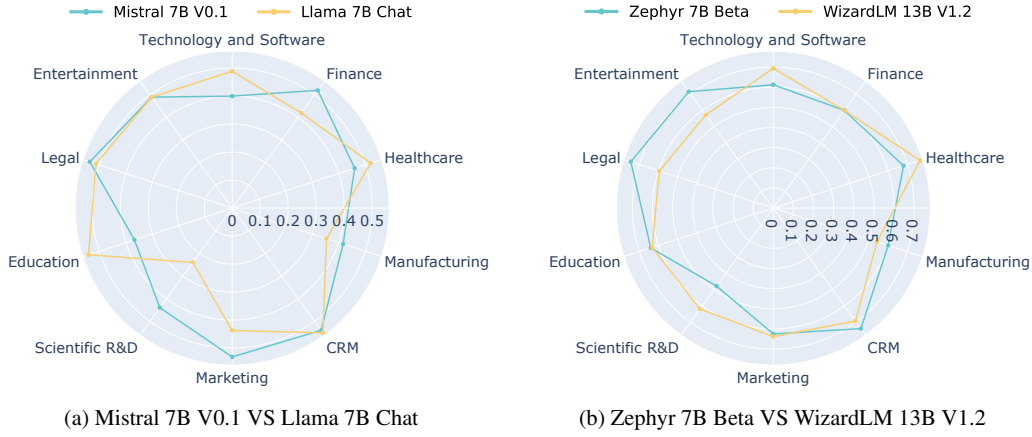


Figure 2: Domain Analysis on different models with similar performance.

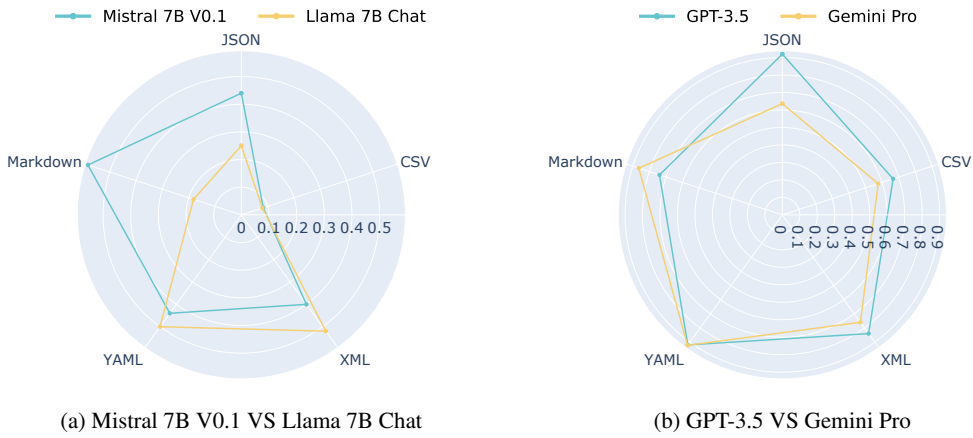


Figure 3: Data format Analysis on different models with similar performance.

format in Commerce and Manufacturing, MathML and Maple format in Scientific R&D, etc. It shows that our benchmark can potentially serve as guidance and probing tools for the choice of domain-specific AI agent foundation models.

Format Analysis. As mentioned in Section 3, except domain-specific formats, FoFo also includes 5 general formats (JSON, XML, CSV, Markdown, and YAML.) and create FORMAT-INSTRU with general formats and domain-specific context. After seeing the performance discrepancy across domains, we are also curious about whether LLMs have their own expertise on different general formats. Therefore in Figure 3, we present the performance comparison of two LLMs achieving similar final accuracy across different general formats. Figure 3(a) shows that both Mistral 7B V0.1 and Llama 7B chat don’t perform well on managing CSV format. While Mistral 7B V0.1 is good at JSON and Markdown and Llama 7B chat expertizes in YAML and XML. Figure 3(b) shows that both GPT-3.5 and Gemini Pro performs well on following YAML format and GPT-3.5 is more specialized on JSON.

It indicates that different models have their own expertise on formats as well. If multiple general formats are suitable for one target generation task, our benchmark can be used to pick the best format for a LLM to reduce errors on format-following on the target task.

Error Analysis. Table 4 reveals that many open-source LLMs underperform on our Format Following (FoFo) benchmark. To understand these shortcomings, we performed an error analysis focusing on Mistral 7B Instruct V0.1 as a representative model. We request human annotators to examine the explanations given by GPT-4 for the failure instances and categorize these reasons into the following groupings: 1) *Incomplete Sections*: The model often neglects essential sections that the prompts mandate, spanning a variety of content areas such as methodological frameworks, data analyses, theoretical discussions. This shortfall is especially pronounced in scientific contexts, as illustrated in Figure 2(a), which depicts Mistral 7B V0.1’s markedly inferior performance in scientific domains. 2) *Incorrect Data Structure*: Mis-

tral 7B V0.1 struggles with adhering to specific structural guidelines, impacting diverse formats including JSON, CSV, Markdown (as highlighted in Figure 3(a)), as well as domain-specific formats such as legal citations and academic referencing. Issues include improper syntax use, inaccurate data structure representations, and failure to follow document layout guidelines. 3) *Missing Detailed Format Requirements*: The model frequently fails to meet specific and detailed formatting requirements. For instances, it presents the "Age" attribute in a text format ("40 years") rather than as a numerical value (40); it ignores the directive to use bullet points ("-") for lists and enumerations; it incorrectly includes headers, despite explicit instructions in the prompt to exclude them.

Human Evaluation Alignment. To evaluate the effectiveness of utilizing GPT-4 as evaluator, we randomly selected 100 annotations made by GPT-4 on the outputs of Mistral 7B V0.1 for a comprehensive human appraisal. We engaged five human experts to review the accuracy of GPT-4’s annotations and conducted an analysis to compare the agreement between human annotators and the GPT-4 evaluator. Our findings revealed that for 84 out of the 100 FORMAT-INSTRU, the evaluations by GPT-4 were in agreement with those of the human experts, yielding an alignment rate of 84%. A closer examination was conducted on the 16 instances where GPT-4’s evaluations diverged from human judgment. It was observed that all these discrepancies were instances of false positives, indicating scenarios where the model’s outputs failed to adhere to specific format requirements, yet were overlooked by GPT-4. For example, one FORMAT-INSTRU required the inclusion of an address, which was absent in the model’s output. Additional errors included the generation of an insufficient number of examples, omission of a required section, and the introduction of non-existent tags. Consequently, these findings suggest that the actual performance of the models might be lower than what is reported by GPT-4’s evaluations.

Taking human evaluations as a benchmark, we infer that the real performance of the models could be approximately 16% lower than the figures presented in Table 4. For example, the accuracy of the Mistral 7B Instruct V0.1 model, as evaluated by GPT-4, stands at 46.91%, but its actual performance is estimated to be around 39.4%. While employing GPT-4 as an evaluator has its limitations, it

significantly reduces the workload associated with human evaluation and provides insights into the comparative performance of different models. For instance, our benchmark indicates that Zephyr 7B Beta is the best open-sourced LLMs in Table 4.

Comparison with IfEval. When generating the FORMAT-INSTRU of FoFo, we prompt GPT-4 to create instructions that contain detailed and complex content generation tasks with specific format configurations, as shown in Figure 1. This is one of the main differences that FoFo has compared to other benchmarks such as IfEval. Although IfEval has a sub-set of prompts that test the format-following capability of LLMs, they are domain-agnostic and contain relatively simple rule-based format configurations. We evaluate representative open-source and closed-source LLMs on IfEval’s detectable format sub set too and find that they achieve much higher accuracy on IfEval’s sub-set compared to on FoFo, as shown in Table 5 in Appendix. It indicates that FoFo is not only the first domain-specific format-following benchmark, but also a much harder one compared to current format-following test sets.

Cost Analysis. In this work, we utilize GPT-4 API for both the creation of our benchmark and the evaluation of LLMs, thereby incurring associated expenses. We estimate the cost of generating the FoFo benchmark to be approximately \$25. Furthermore, the expense of evaluating a single LLM on FoFo is estimated to be around \$40. In future endeavors, we plan to consider using GPT-4-Turbo as the evaluator to reduce costs. Please refer to Appendix A.2 for more details.

5 Conclusion

In conclusion, our introduction of FoFo marks a significant advancement in evaluating large language models’ (LLMs) format-following capabilities, a crucial but previously overlooked aspect. Through a novel AI-Human collaborative construction, FoFo offers a comprehensive benchmark covering a diverse range of formats and instructions. Our findings reveal that format-following is an independent skill set not correlated with content generation performance, highlight a gap between open and closed-source LLMs in format adherence, and underscore the variability of LLMs’ format-following proficiency across domains.

Limitations

Our research marks a significant step forward in assessing the format-following capabilities of large language models (LLMs). However, it is not without its challenges. One constraint is our reliance on human experts for benchmark validation, including test prompt verification. This dependence could both introduce subjectivity and limit the ability to scale. To address this, future work will aim to reduce human involvement by crafting a more automated, yet equally robust, system for test case generation and validation, thereby broadening the benchmark’s applicability. Moreover, the evaluation of LLMs, particularly through the use of GPT-4 APIs, incurs some costs. An alternative we intend to explore is employing GPT-4 Turbo as a more cost-effective solution without compromising the assessment’s quality. Furthermore, our benchmark, although extensive, might not fully represent the diverse array of format requirements seen in real-world settings. Continuous refinement and expansion of our benchmark are essential to more accurately reflect the vast spectrum of practical use cases, enhancing its utility for future LLM development and deployment.

References

- Rohan Anil, Andrew M Dai, Orhan Firat, Melvin Johnson, Dmitry Lepikhin, Alexandre Passos, Siamak Shakeri, Emanuel Taropa, Paige Bailey, Zhifeng Chen, et al. 2023. Palm 2 technical report. *arXiv preprint arXiv:2305.10403*.
- Rishabh Bhardwaj and Soujanya Poria. 2023. Red-teaming large language models using chain of utterances for safety-alignment. *arXiv preprint arXiv:2308.09662*.
- Yihan Chen, Benfeng Xu, Quan Wang, Yi Liu, and Zhendong Mao. 2024. Benchmarking large language models on controllable generation under diversified instructions. *arXiv preprint arXiv:2401.00690*.
- Wei-Lin Chiang, Zhuohan Li, Zi Lin, Ying Sheng, Zhanghao Wu, Hao Zhang, Lianmin Zheng, Siyuan Zhuang, Yonghao Zhuang, Joseph E. Gonzalez, Ion Stoica, and Eric P. Xing. 2023. Vicuna: An open-source chatbot impressing gpt-4 with 90%* chatgpt quality. <https://lmsys.org/blog/2023-03-30-vicuna/>.
- Hyung Won Chung, Le Hou, Shayne Longpre, Barret Zoph, Yi Tay, William Fedus, Yunxuan Li, Xuezhi Wang, Mostafa Dehghani, Siddhartha Brahma, et al. 2022. Scaling instruction-finetuned language models. *arXiv preprint arXiv:2210.11416*.
- Jan Clusmann, Fiona R Kolbinger, Hannah Sophie Muti, Zunamys I Carrero, Jan-Niklas Eckardt, Narmin Ghaffari Laleh, Chiara Maria Lavinia Löffler, Sophie-Caroline Schwarzkopf, Michaela Unger, Gregory P Veldhuizen, et al. 2023. The future landscape of large language models in medicine. *Communications Medicine*, 3(1):141.
- Jiayi Cui, Zongjian Li, Yang Yan, Bohua Chen, and Li Yuan. 2023. Chatlaw: Open-source legal large language model with integrated external knowledge bases. *arXiv preprint arXiv:2306.16092*.
- Robert H Dolin, Liora Alschuler, Calvin Beebe, Paul V Biron, Sandra Lee Boyer, Daniel Essin, Elliot Kimber, Tom Lincoln, and John E Mattison. 2001. The hl7 clinical document architecture. *Journal of the American Medical Informatics Association*, 8(6):552–569.
- Yann Dubois, Xuechen Li, Rohan Taori, Tianyi Zhang, Ishaan Gulrajani, Jimmy Ba, Carlos Guestrin, Percy Liang, and Tatsunori B Hashimoto. 2023. AlpacaFarm: A simulation framework for methods that learn from human feedback. *arXiv preprint arXiv:2305.14387*.
- Zhiwei Fei, Xiaoyu Shen, Dawei Zhu, Fengzhe Zhou, Zhuo Han, Songyang Zhang, Kai Chen, Zongwen Shen, and Jidong Ge. 2023. Lawbench: Benchmarking legal knowledge of large language models. *arXiv preprint arXiv:2309.16289*.
- Chen Gao, Xiaochong Lan, Zhihong Lu, Jinzhu Mao, Jinghua Piao, Huandong Wang, Depeng Jin, and Yong Li. 2023. S³: Social-network simulation system with large language model-empowered agents. *arXiv preprint arXiv:2307.14984*.
- Gemini Team Google. 2023. Gemini: a family of highly capable multimodal models. *arXiv preprint arXiv:2312.11805*.
- Neel Guha, Julian Nyarko, Daniel E Ho, Christopher Ré, Adam Chilton, Aditya Narayana, Alex Chohlas-Wood, Austin Peters, Brandon Waldon, Daniel N Rockmore, et al. 2023. Legalbench: A collaboratively built benchmark for measuring legal reasoning in large language models. *arXiv preprint arXiv:2308.11462*.
- Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. 2020. Measuring massive multitask language understanding. In *International Conference on Learning Representations*.
- Albert Q Jiang, Alexandre Sablayrolles, Arthur Mensch, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Florian Bressand, Gianna Lengyel, Guillaume Lample, Lucile Saulnier, et al. 2023. Mistral 7b. *arXiv preprint arXiv:2310.06825*.
- Cong Jiang and Xiaolei Yang. 2023. Legal syllogism prompting: Teaching large language models for legal

- judgment prediction. In *Proceedings of the Nineteenth International Conference on Artificial Intelligence and Law*, pages 417–421.
- Stephanie Lin, Jacob Hilton, and Owain Evans. 2022. Truthfulqa: Measuring how models mimic human falsehoods. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 3214–3252.
- Zilin Ma, Yiyang Mei, and Zhaoyuan Su. 2023. Understanding the benefits and challenges of using large language model-based conversational agents for mental well-being support. In *AMIA Annual Symposium Proceedings*, volume 2023, page 1105. American Medical Informatics Association.
- Vinod Muthusamy, Yara Rizk, Kiran Kate, Praveen Venkateswaran, Vatche Isahagian, Ashu Gulati, and Parijat Dube. 2023. Towards large language model-based personal agents in the enterprise: Current trends and open problems. In *Findings of the Association for Computational Linguistics: EMNLP 2023*, pages 6909–6921.
- OpenAI. 2023a. Chatgpt. <https://openai.com/chatgpt>.
- OpenAI. 2023b. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774*.
- Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. 2022. Training language models to follow instructions with human feedback. *Advances in Neural Information Processing Systems*, 35:27730–27744.
- Nigam H Shah, David Entwistle, and Michael A Pfeffer. 2023. Creation and adoption of large language models in medicine. *Jama*, 330(9):866–869.
- Mohit Shridhar, Xingdi Yuan, Marc-Alexandre Côté, Yonatan Bisk, Adam Trischler, and Matthew Hausknecht. 2020. Alfworld: Aligning text and embodied environments for interactive learning. *arXiv preprint arXiv:2010.03768*.
- Chenglei Si, Navita Goyal, Sherry Tongshuang Wu, Chen Zhao, Shi Feng, Hal Daumé III, and Jordan Boyd-Graber. 2023. Large language models help humans verify truthfulness—except when they are convincingly wrong. *arXiv preprint arXiv:2310.12558*.
- Liyan Tang, Zhaoyi Sun, Betina Idnay, Jordan G Nestor, Ali Soroush, Pierre A Elias, Ziyang Xu, Ying Ding, Greg Durrett, Justin F Rousseau, et al. 2023. Evaluating large language models on medical evidence summarization. *npj Digital Medicine*, 6(1):158.
- Arun James Thirunavukarasu, Darren Shu Jeng Ting, Kabilan Elangovan, Laura Gutierrez, Ting Fang Tan, and Daniel Shu Wei Ting. 2023. Large language models in medicine. *Nature medicine*, 29(8):1930–1940.
- Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, et al. 2023. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288*.
- Lewis Tunstall, Edward Beeching, Nathan Lambert, Nazneen Rajani, Kashif Rasul, Younes Belkada, Shengyi Huang, Leandro von Werra, Clémentine Fourrier, Nathan Habib, et al. 2023. Zephyr: Direct distillation of lm alignment. *arXiv preprint arXiv:2310.16944*.
- Guan Wang, Sijie Cheng, Xianyuan Zhan, Xiangang Li, Sen Song, and Yang Liu. 2023. Openchat: Advancing open-source language models with mixed-quality data. *arXiv preprint arXiv:2309.11235*.
- Yizhong Wang, Yeganeh Kordi, Swaroop Mishra, Alisa Liu, Noah A Smith, Daniel Khashabi, and Hananeh Hajishirzi. 2022. Self-instruct: Aligning language model with self generated instructions. *arXiv preprint arXiv:2212.10560*.
- Jason Wei, Maarten Bosma, Vincent Y Zhao, Kelvin Guu, Adams Wei Yu, Brian Lester, Nan Du, Andrew M Dai, and Quoc V Le. 2021. Finetuned language models are zero-shot learners. *arXiv preprint arXiv:2109.01652*.
- Zhiheng Xi, Wenxiang Chen, Xin Guo, Wei He, Yiwen Ding, Boyang Hong, Ming Zhang, Junzhe Wang, Senjie Jin, Enyu Zhou, et al. 2023. The rise and potential of large language model based agents: A survey. *arXiv preprint arXiv:2309.07864*.
- Can Xu, Qingfeng Sun, Kai Zheng, Xiubo Geng, Pu Zhao, Jiazhan Feng, Chongyang Tao, and Daxin Jiang. 2023. Wizardlm: Empowering large language models to follow complex instructions. *arXiv preprint arXiv:2304.12244*.
- John Yang, Akshara Prabhakar, Shunyu Yao, Kexin Pei, and Karthik R Narasimhan. 2023. Language agents as hackers: Evaluating cybersecurity skills with capture the flag. In *Multi-Agent Security Workshop@NeurIPS’23*.
- Zhexin Zhang, Leqi Lei, Lindong Wu, Rui Sun, Yongkang Huang, Chong Long, Xiao Liu, Xuanyu Lei, Jie Tang, and Minlie Huang. 2023. Safety-bench: Evaluating the safety of large language models with multiple choice questions. *arXiv preprint arXiv:2309.07045*.
- Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric Xing, et al. 2023a. Judging llm-as-a-judge with mt-bench and chatbot arena. *arXiv preprint arXiv:2306.05685*.
- Ou Zheng, Mohamed Abdel-Aty, Dongdong Wang, Chenzhu Wang, and Shengxuan Ding. 2023b. Traffic-safetygpt: Tuning a pre-trained large language model to a domain-specific expert in transportation safety. *arXiv preprint arXiv:2307.15311*.

Jeffrey Zhou, Tianjian Lu, Swaroop Mishra, Siddhartha Brahma, Sujoy Basu, Yi Luan, Denny Zhou, and Le Hou. 2023a. Instruction-following evaluation for large language models. *arXiv preprint arXiv:2311.07911*.

Shuyan Zhou, Frank F Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Yonatan Bisk, Daniel Fried, Uri Alon, et al. 2023b. Webarena: A realistic web environment for building autonomous agents. *arXiv preprint arXiv:2307.13854*.

A Appendix A

A.1 Evaluated Models

We evaluate the following models with our FoFo benchmark:

Open-source LLMs

- Vicuna 13B V1.3 (Chiang et al., 2023)²
- Vicuna 13B V1.5-16k (Chiang et al., 2023)³
- WizardLM 13B V1.1 (Xu et al., 2023)⁴
- WizardLM 13B V1.2 (Xu et al., 2023)⁵
- Openchat V3.2-super (Wang et al., 2023)⁶
- Llama 2 7B Chat (Touvron et al., 2023)⁷
- Llama 2 13B Chat (Touvron et al., 2023)⁸
- Mistral 7B Instruct V0.1 (Jiang et al., 2023)⁹
- Zephyr 7B Beta (Tunstall et al., 2023)¹⁰

Closed-source LLMs

- GPT-3.5 (OpenAI, 2023a)¹¹
- GPT-4 (OpenAI, 2023b)¹²
- Gemini-Pro (Google, 2023)¹³
- PaLM 2 for Text 32k (Anil et al., 2023)¹⁴

²<https://huggingface.co/lmsys/vicuna-13b-v1.3>

³<https://huggingface.co/lmsys/vicuna-13b-v1.5-16k>

⁴<https://huggingface.co/WizardLM/WizardLM-13B-V1.1>

⁵<https://huggingface.co/WizardLM/WizardLM-13B-V1.2>

⁶https://huggingface.co/openchat/openchat_v3.2_super

⁷<https://huggingface.co/meta-llama/Llama-2-7b-chat-hf>

⁸<https://huggingface.co/meta-llama/Llama-2-13b-chat-hf>

⁹<https://huggingface.co/mistralai/Mistral-7B-Instruct-v0.1>

¹⁰<https://huggingface.co/HuggingFaceH4/zephyr-7b-beta>

¹¹We use “gpt-4” <https://platform.openai.com/docs/models/gpt-4-and-gpt-4-turbo>

¹²We use “gpt-3.5-turbo-1106”. <https://platform.openai.com/docs/models/gpt-3-5-turbo>

¹³<https://cloud.google.com/vertex-ai/docs/generative-ai/model-reference/gemini>

¹⁴We use “text-bison-32k”. <https://cloud.google.com/vertex-ai/docs/generative-ai/model-reference/text>

A.2 Cost Analysis

In this study, we utilize GPT-4 for both the construction of our benchmark and the evaluation of LLMs. This approach incurs certain costs associated with utilizing the GPT-4 API. The cost is determined by the number of tokens in both the input and output, with pricing set at \$0.03 per 1,000 input tokens and \$0.06 per 1,000 output tokens¹⁵. Given that one token approximately equates to four characters of common English text according to OpenAI’s guidelines¹⁶, we can estimate the average number of tokens based on the character count.

In the process of developing our benchmark, the primary expenditure arises during the creation of FoFo, as detailed in Section 3.1. Analysis reveals that the mean length of input characters is 818, translating to approximately 205 GPT-4 tokens. Conversely, the output character count averages at 2908, equivalent to about 727 GPT-4 tokens. This setup yields an average cost per prompt of $\$0.03 * 0.205$ (for the input) + $\$0.06 * 0.727$ (for the output), amounting to \$0.05. Considering the necessity to craft 500 such prompts, the aggregate cost allocated for the generation of test prompts stands at \$25.

In our evaluation of various LLMs using GPT-4, the inputs consist of three components as detailed in Figure 7: the length of the evaluation template (which contains 1,594 characters), the length of FORMAT-INSTRU, and the length of the model outputs. As indicated in Table 3, the average character length of FORMAT-INSTRU is 2,908. The average character length of models’ outputs is 4,163. Consequently, the average input character length for evaluation purposes totals 8,665, equivalent to approximately 2,166 tokens for GPT-4. This results in an average input cost of: $\$0.03 * 2.166 = \0.065 . For the outputs, the average character length during evaluation is 1,098, translating to roughly 275 tokens for GPT-4. This leads to an average output cost of: $\$0.06 * 0.275 = \0.0165 . Therefore, the average cost of evaluating one LLM using a single prompt stands at \$0.0815. The entire Format Following (FoFo) benchmark comprises around 500 prompts, culminating in a total evaluation cost of approximately \$40 for one LLM across the entire benchmark. In future endeavors, we are considering the adoption of GPT-4-Turbo for evaluation purposes to further mitigate costs.

¹⁵<https://openai.com/pricing>

¹⁶<https://platform.openai.com/tokenizer>

Model	FoFo	IfEval
	Format Acc	Acc on Detectable Formats
WizardLM 13B V1.1 (Xu et al., 2023)	27.00	59.24
Mistral 7B Instruct V0.1 (Jiang et al., 2023)	46.91	60.51
WizardLM 13B V1.2 (Xu et al., 2023)	63.54	69.43
Zephyr 7B Beta (Tunstall et al., 2023)	64.12	66.24
GPT-3.5 (OpenAI, 2023a)	80.66	91.72

Table 5: Comparison of LLMs’ performance on IfEval format sub-set and on FoFo.

A.3 Comparison with IfEval.

Table 5 illustrates the performance of representative open-source and closed-source LLMs on FoFo and the detectable formats subcategory in IfEval. As shown in Table 5, these models achieve significantly higher accuracy on the IfEval subset compared to FoFo. Furthermore, the performance gap between different models on IfEval is much narrower compared to that on FoFo.

A.4 Examples of FORMAT-INSTRU

We show more examples of FORMAT-INSTRU in Figure 4 and Figure 5. Figure 4 shows an example when “Domain” = “Commerce and Manufacturing”, “Subdomain” = “Manufacturing Process Optimization”, and “Format” = “Standard Operating Procedure (SOP)”, while Figure 5 is an example when “Domain” = “Education”, “Subdomain” = “Automated Grading Systems”, and “Format” = “Markdown”. Both examples include detailed enough format specifications under each domain.

A.5 Examples of GPT-4 annotations

We also list an example of GPT-4 annotations on the output from Mistral 7B Instruct V0.1 in Figure 6. In this example, the format correctness of the output from Mistral 7B Instruct V0.1, based on a given instruction format, is labeled as ‘0’ (indicating ‘False’). GPT-4 also includes the reasons why the format of the output is not correct. For example, the date and time stamp is not correctly formatted or the correct software name is not used.

A.6 List of domain specific data formats

The full list of domain specific data formats is illustrated in page 16-18.

A.7 Evaluation Prompt Template

The evaluation prompt template we used to evaluate the performance of different models is shown in Figure 7.

Domain	Mistral-7b-inst		Llama-2-7b	
	mean	var	mean	var
Healthcare	0.2	0.16	0.8	0.16
Finance	0.4	0.24	0.4	0.24
Technology and Software	0.6	0.24	0.4	0.24
Commerce and Manufacturing	0.4	0.24	0.5	0.25
CRM	0.6	0.24	0.4	0.24
Marketing	0.4	0.24	0.2	0.16
Scientific R&D	0.6	0.24	0	0
Education	0.2	0.16	0.6	0.24
Legal	0	0	0.4	0.24
Arts and Entertainment	0.6	0.24	0.4	0.24

Table 6: Domain Analysis within the same data format.

A.8 Domain analysis within the same data format

To decouple the performance analysis for domains and data formats, we present a domain analysis within the same data format for two distinct models: Mistral 7B V0.1 and Llama 7B Chat. In Figure 3(a), we can observe that the performance of Mistral 7B V0.1 and Llama 7B Chat is quite similar when considering the YAML format.

As shown in the table 6, we discover that even within the same YAML data format, the performance of these models varies significantly across different domains. For instance, in the health domain, Mistral 7B V0.1 achieves a mean accuracy of 0.2 with a variance of 0.16. Conversely, Llama 7B Chat performs well in the health domain, boasting a mean accuracy of 0.8 with a variance of 0.16. On the other hand, when it comes to the domain of Scientific Research and Development, Mistral 7B V0.1 achieves a mean accuracy of 0.6 with a variance of 0.24. In contrast, Llama 7B Chat disappoints in this particular domain, exhibiting a mean accuracy of 0 with a variance of 0.

Generate a comprehensive SOP for the optimization of the injection molding process to increase the efficiency of production while maintaining product quality. The SOP should be titled \"Injection Molding Process Optimization\" and should be divided into the following sections and sub-sections with detailed instructions and parameters, adhering to the standard ISO 9001:2015 for quality management systems.

You are allowed to create plausible dummy data where specific data is not provided. Use metric units for all measurements. Ensure that each step is written in imperative mood (command voice) for clarity and adherence to the SOP format.

1. Document Control

- 1.1. Document Information
 - Title
 - Document ID
 - Version Number
 - Creation Date
 - Last Review Date
 - Next Review Due
- 1.2. Revision History
 - Table with columns for Revision Number, Date, Description of Changes, and Changed By
- 1.3. Document Approval
 - Names and Signatures of SOP Author and Quality Assurance Manager

2. Scope

- Define the boundaries of the process optimizations, including the types of products and machinery to which this SOP applies.

3. Definitions and Abbreviations

- Provide a list of all relevant terms and their definitions, as well as any abbreviations used in the document.

4. Responsibilities

- 4.1. Management
 - Assign roles and responsibilities to management staff involved in the process optimization.
- 4.2. Machine Operators
 - Define the duties and tasks of machine operators in implementing the SOP.
- 4.3. Quality Assurance Personnel
 - Outline the oversight and compliance roles of quality assurance staff.

5. Equipment and Materials

- List all equipment and materials required for the injection molding process, including any specific models or brands.

6. Process Optimization Procedure

- 6.1. Pre-Optimization Analysis
 - Instructions on assessing current process performance and identifying areas for improvement.
- 6.2. Design of Experiments (DOE)
 - Step-by-step approach for planning and conducting experiments to determine optimal process parameters.
- 6.3. Validation Procedure
 - Guidance on verifying that adjustments lead to the desired improvements without compromising quality.
- 6.4. Implementation
 - Detailed description of how to carry out process changes on the production floor.
- 6.5. Monitoring and Control
 - Procedures for monitoring process performance post-implementation and criteria for ongoing control.

7. Documentation and Records

- Specify the types and formats of records to be kept as evidence of process optimization efforts and results.

8. Safety Consideration

- List any safety protocols that must be followed during the optimization process.

9. Contingency Plans

- Outline steps to be taken if process modifications do not yield expected results or if new issues arise.

Remember to ensure all instructions promote consistent production and are adaptable to slight variations in machine or material properties. Use clear language to avoid misinterpretation and ensure repeatability of process enhancements.

Figure 4: An FORMAT-INSTRU example when “Domain” = “Commerce and Manufacturing”, “Subdomain” = “Manufacturing Process Optimization”, and “Format” = “Standard Operating Procedure (SOP)”.

Design and detail a comprehensive blueprint for an Automated Essay Grading System in the education sector. The documentation must be formatted in Markdown, and should follow these specifics:

1. **Header structure**:

- Title (## Title): \"Automated Essay Grading System (AEGS) Design Document\"
- Introduction (### Introduction)
- System Overview (### System Overview)
- Technical Architecture (### Technical Architecture)
- Data Collection (#### Data Collection)
- Machine Learning Model (#### Machine Learning Model)
- Natural Language Processing (#### Natural Language Processing)
- Implementation Plan (### Implementation Plan)
- Phase 1: Proof of Concept (#### Phase 1: Proof of Concept)
- Phase 2: Pilot Testing (#### Phase 2: Pilot Testing)
- Phase 3: Full-scale Deployment (#### Phase 3: Full-scale Deployment)
- Evaluation Criteria (### Evaluation Criteria)
- Limitations and Ethical Considerations (### Limitations and Ethical Considerations)
- Conclusion (### Conclusion)

2. **Formatting Guidelines**:

- **Bold** for section headings and subheadings.
- *Italicize* any terms that are not generally known.
- Inline ``code`` format should be used when referencing specific variables or code snippets related to the grading algorithm.
- Bulleted lists for features or requirements.
- Numbered lists for step-by-step guides or processes.
- Blockquotes for large quotations or excerpts from academic papers.
- Hyperlinks for all external references - ensure that they are formatted as [anchor text](URL).

3. **Content Specifics**:

- **Introduction**: Provide a brief overview of the need for automated essay grading systems.
- **System Overview**: Explain the general working mechanism for the AEGS.
- **Technical Architecture**: Include a detailed description using the following structure:
- **Data Collection**: Describe the sources and types of data needed.
- **Machine Learning Model**: Define the model to be used and justify the choice.
- **Natural Language Processing**: Outline the NLP techniques that will be applied in grading essays.
- **Implementation Plan**: Elaborate on the three phases of implementation, including timelines, required resources, and milestones.
- **Evaluation Criteria**: Set forth the metrics on how the essays will be evaluated by the system.
- **Limitations and Ethical Considerations**: Discuss any potential biases, privacy concerns, and mitigation strategies.
- **Conclusion**: Summarize the system's potential impact on the educational landscape.

4. **Imaginary Data**:

Feel free to create imaginary datasets, studies, or references where necessary, ensuring they are realistic and consistent within the scope of automated grading systems (e.g., dataset sizes, types of essays, performance metrics).

5. **Code Snippets**:

Include a few Python pseudo-code snippets as examples for how data preprocessing or feature extraction might be done in the system. Ensure proper Markdown code block formatting with syntax highlighting as follows:

```
```python
This is a Python code snippet example
def preprocess_text(text):
 # Code to preprocess text
 pass
```
```

The final document should be comprehensive, technically detailed, and elegantly formatted such that it can be directly used as a formal proposal for building an Automated Essay Grading System. In cases where you lack context or data, feel free to improvise and include plausible assumptions.

Figure 5: An FORMAT-INSTRU example when “Domain” = “Education”, “Subdomain” = “Automated Grading Systems”, and “Format” = “Markdown”.

Model Output



| Domain | Subdomain | Dataformat |
|-------------------------|------------------------------------|---|
| Healthcare | Medical Diagnostics | ICD-10 format (International Classification of Diseases 10th Revision); LOINC format (Logical Observation Identifiers Names and Codes); CAP format (College of American Pathologists protocol); DICOM SR (Structured Report); HL7 CDA (Health Level Seven Clinical Document Architecture) |
| | Medical Treatment | Medical Reports; Prescription Formats; SOAP Notes; Discharge Summary; Clinical Trial Protocols |
| | Patient Care Management | Electronic Health Record (EHR) Format; Discharge Summary Format; Clinical Trial Report Format; Prescription Format; Medical Coding and Billing Statement Format |
| | Clinical Trial Analysis | Clinical Study Report (CSR); Clinical Trial Protocol; Patient Reported Outcomes (PRO); Patient Data Report (PDR); Adverse Event Report |
| | Pharmaceuticals | RxNorm Format; HL7 (Health Level 7) Format; ICD-10 Format; LOINC Format; Fast Healthcare Interoperability Resources (FHIR) Format |
| Finance | Fraud Detection | Compromised Account Report Format; Credit Card Fraud Alert Format; Investment Fraud Detection Report; Loan Fraud Report Format; Insurance Fraud Detection Report |
| | Algorithm Trading | FIX Protocol Message; Standard Portfolio Analysis of Risk Data; Thomson Reuters EIKON data; Bloomberg Terminal Data; Morningstar Data |
| | Personalized Financial Advice | Financial Reports; Investment Strategy Reports; Personal Financial Plans; Risk Profile Reports; Asset Performance Reports |
| | Risk management | Financial Risk Analysis Reports; Basel III Regulatory Filings; Value At Risk (VaR) Statements; Credit Default Swap (CDS) Spreads; Stress Testing Reports |
| | Regulatory Compliance | Legal Document Format; Financial Report Format (FRF); Risk Assessment Reports; Regulatory Filings Format; Compliance Audit Reports |
| Technology and Software | Web Design | HTML (HyperText Markup Language); CSS (Cascading Style Sheets); JavaScript; SVG (Scalable Vector Graphics); .htaccess |
| | Programming | Python Files (.py); JavaScript Files (.js); SQL Files (.sql); Java Source Files (.java); C++ Source Code Files (.cpp) |
| | UI/UX Design | UX/UI Mockup Annotation; User Journey Mapping Text; Usability Test Session Transcripts; UX/UI Design Specification (Typography Palette etc.); User Persona Description |
| | Data Analysis | SQL Result Set; Python Pandas DataFrame; R Data Frame Output; Data Dictionary Output; Log File Output |
| | Testing | Test Plan; Test Case Description; Bug/Issue Report; Test Summary Report; Requirement Traceability Matrix (RTM) |
| | E-commerce Personalization | E-commerce Invoice Format; Personalized Product Recommendation; Shopping Cart Abandonment Reminder; Shipping and Delivery Notification; Customer Review and Rating Format |
| | Manufacturing Process Optimization | Manufacturing Reports Format (MRF); Bill of Materials (BOM); Work Instruction Format (WIF); Standard Operating Procedure (SOP); Production Scheduling Format (PSF) |

Continued on next page

| Domain | Subdomain | Dataformat |
|--|---|---|
| Commerce and Manufacturing | Inventory and Supply Chain Management | Purchase Order (PO) Format; Inventory Report Format; Sales Forecast Format; Shipping Status Format; Return/Replacement Order Format |
| | Quality Control | Product Inspection Report; Quality Assurance (QA) Test Report; Defect Tracking Log; Product Compliance Certificate; Supplier Quality Report |
| | Smart Logistics and Route Optimization | Freight Bill Format; Inventory Update Format; Shipping Manifest Format; Route Optimization Report; Order Pick List |
| Customer Relationship Management (CRM) | Customer Service | Customer Email Response Format; Live Chat Transcript Format; Customer Feedback Form Response Format; Ticketing System Response Format; Social Media Comment Response Format |
| | Sales Forecasting | Sales Forecast Report; Key Performance Indicator (KPI) Report; Pipeline Report; Sales Targets Report; CRM Dashboard Summary |
| | Recruitment Assistants | Resume Format; Job Description Format; Interview Schedule Format; Applicant Status Update Format; Candidate Comparative Analysis |
| | Project Management | Gantt Chart Representation; Task Breakdown Structure (TBS); Project Status Reports; Meeting Minutes; Risk Assessment Reports |
| | Lead Scoring | Lead Score Reports; Sales Funnel Analysis Documents; Lead Demographic Profiles; Customer Interaction Logs; Predictive Scoring Reports |
| Marketing | Consumer Behavior Analysis | Consumer Behavior Report; Marketing Performance Dashboard; Advertising Copy Feedback; Social Media Sentiment Analysis; Competitor Analysis Summary |
| | Advertising Campaign Optimization | Advertising Audience Profile Format; KPI Report Format; A/B Test Result Format; Competitive Analysis Format; Campaign Budget Format |
| | Content Curation and Creation | Blog Post; Social Media Post; Email Newsletters; Press Release; SEO Content |
| | Social Media Management | Social Media Report; Content Calendar; User Sentiment Analysis; Hash-tags Usage Report; Social Media Customer Inquiries Response |
| | Search Engine Optimization | SERP (Search Engine Results Page) Report; SEO Keyword Analysis; On-Page SEO Audits; Backlink Profile Report; Competitor Analysis Report |
| Scientific Research and Development | Mathematical Research | LaTeX; MathML (Mathematical Markup Language); SageMath Notebooks; Maple; MATLAB scripts |
| | Physics | LaTeX (.tex); MathML (.mathml); BibTeX (.bib); Research Paper Abstract Structured Text; Physical Quantities and Units in UCUM (.ucum) |
| | Chemistry and Biological Sciences | FASTA Format; PDB Format (Protein Data Bank); GenBank Format; SMILES Format (Simplified Molecular Input Line Entry System); MOL and SDF Formats (MOlecular and Structure-Data File) |
| | Environmental Sciences and Climate Change | Research Paper (APA Format); Scientific Report Format; Environmental Impact Statement (EIS); Peer Review Reports; Policy Briefs |

Continued on next page

| Domain | Subdomain | Dataformat |
|------------------------|---------------------------------------|---|
| Education | Space Exploration | NASA Planetary Data System (PDS) Format; MISSION OPERATION REPORT (MOR) Format; OBSerVation Time Series (OBSErVTS) Format; Spacecraft Event Language (SEL) Format; Telescope Observation Request (TOR) Format |
| | Adaptive Learning Platforms | Personalized Learning Plan (PLP); Assessment Results Format (ARF); Interactive Course Content Format (ICCF); Collaboration Log Format (CLF); Task Progress Report Format (TPRF) |
| | Intelligent Tutoring Systems | Lesson Summary Format; Student Performance Report Format; Quiz/Instruction Format; Feedback/Correction Format; Personalized Learning Path Recommendation Format |
| | Automated Grading Systems | Rubric Score Format; Student Report Format; Class Rank Format; Question Assessment Format; Error Analysis Format |
| | Education Data Analysis | Report in Academic Results Format (ARF); Student Behavior Analysis Format (SBAF); Educational Content Analysis Format (ECAf); Learning Style Analysis Format (LSAF); Teaching Performance Evaluation Format |
| | Language Learning | Learning Material Format; Quizzes/Test Format; Progress Report Format; Language Translation Format; Phonetic Script Format |
| Legal | Contract Review and Analysis | Legal Brief Format; Contract Abstract Format; Risk Assessment Format; Clause Breakdown Format; Legal Opinion Format |
| | Legal Document Automation | Legal XML (LegalXML); Interactive Legal Applications Markup Language (iLAML) |
| | Legal Research | Legal Brief; Case Citation; Contract Format; Statute |
| | Predictive Legal Analytics | Legal Reporting Document; Case Brief Format; Legal Opinion Letter Format; Legal Case Study Format |
| | Intellectual Property (IP) Management | Patent Disclosure Forms; Trademark Registration Documents; Copyright Registration Forms; Intellectual Property Agreement Contracts; Patent Litigation Documents |
| Arts and Entertainment | Music | Lyrics Text Format; Chord Sheet Format; Tracklist Format; Metadata Format |
| | Film Scriptwriting | Screenplay Format; Synopsis/Outline Format; Treatment Format; Beat Sheet Format; Character Profile/Backstory Format |
| | Visual Art Creation | SVG (Scalable Vector Graphics); GLSL (OpenGL Shading Language); TikZ (code for creating vector graphics); POV-Ray Scene Description Language (SDL); G-code (a language in which people instruct computerized machine tools) |
| | Video Game Development | Game Design Document (GDD); Interactive Fiction Markup Language (IFML); Lua table for game configuration; GLSL Shader Code; Unreal Engine Blueprints Visual Scripting (Print String Node) |
| | Sports Analytics and Performance | Game Statistics Report; Training Performance Summary; Player Ranking Report; Injury Report; Match Prediction Report |

| Evaluate Prompt Template |
|---|
| <pre> <lim_start>system You are a helpful assistant who evaluates the correctness and quality of models' outputs. <lim_end> <lim_start>user I would like you to create a leaderboard that evaluates the correctness of the format of answers from various large language models. To accomplish this, you will need to analyze the text prompts given to the models and their corresponding answers. Specifically, please ensure that your evaluation outputs are properly formatted as a json string. I will provide both the prompts and the responses for this purpose. Here is the prompt: { "instruction": "{instruction}" } Here are the outputs of the models: [{ "model": "model", "answer": "{output}" },] Please evaluate the formatting of the model's responses by checking if they comply with the format specifications stated in the prompt. Perform a thorough format check and provide a detailed explanation for why the format is correct or incorrect. Your feedback should include the name of the model, followed by the format correctness status represented as '1' for correct and '0' for incorrect. Present your reasoning as bullet points within a single string for each model assessed. In other words, you should produce the following output: “json [{ "model": <model-name>, "format_correctness": <correctness>, "reasons": <reasons-of-format-correctness> }]” Please note that your response should be a properly formatted JSON string and should not contain any additional content. We will load it directly as a JSON string in Python. <lim_end> </pre> |

Figure 7: Evaluate template prompt.